GRIFFITH COLLEGE DUBLIN

Assignment Cover Sheet

Student name: Toluwani Aderibigbe

Student number: 3050667

Faculty: **Computing Science**

Course: BSCH **Stage/year:** **4**

Subject: Cloud Services and platforms

Study Mode: Full time **yes** Part-time _____

Lecturer Name: Sheresh Zahoor

Assignment Title: Assignment 2

No. of pages: _____

Disk included? Yes _____ No _____

Additional Information: (ie. number of pieces submitted, size of assignment, A2, A3 etc)

Date due: 3/05/2026

Date submitted: 1/05/2026

Plagiarism disclaimer:

I understand that plagiarism is a serious offence and have read and understood the college policy on plagiarism. I also understand that I may receive a mark of zero if I have not identified and properly attributed sources which have been used, referred to, or have in any way influenced the preparation of this assignment, or if I have knowingly allowed others to plagiarise my work in this way.

I hereby certify that this assignment is my own work, based on my personal study and/or research, and that I have acknowledged all material and sources used in its preparation. I also certify that the assignment has not previously been submitted for assessment and that I have not copied in part or whole or otherwise plagiarised the work of anyone else, including other students.

Signed: _____Ade_____ **Date: ___1/05/2026_____**

Please note: *Students **MUST** retain a hard / soft copy of **ALL** assignments as well as a receipt issued and signed by a member of Faculty as proof of submission.*

Table of Contents

1. Introduction 2
2. Overall Approach 2
3. System Architecture 3
    - 3.1 Frontend 3
    - 3.2 Backend (FastAPI) 3
    - 3.3 MongoDB 4
    - 3.4 Azure Blob Storage 4
4. Authentication 5
5. Key Functionalities 6
    - 5.1 Folder Management 6
    - 5.2 File Upload 6
    - 5.3 Overwrite Protection 6
    - 5.4 File Download 6
    - 5.5 File Deletion 7
    - 5.6 Duplicate Detection 7
    - 5.7 Metadata Display 7
    - 5.8 Search and Sorting 7
6. System Workflow 7
7. Challenges Faced 8
    - 7.1 Token Expiration 8
    - 7.2 Azure Storage Setup 8
    - 7.3 Duplicate Detection 9
    - 7.4 Overwrite Handling 9
8. Refrences 9
9. Conclusion 10
10. Github repository 10

Cloud File Manager – Final Documentation

1. Introduction

The main goal of my application is to allow users to store, organize, and manage files using cloud technologies. Users should be able to create folders, upload and download files, detect duplicates, and search through stored data.

I built the application using FastAPI as the backend framework, MongoDB for storing structured data, Firebase Authentication for managing user login, and Azure Blob Storage for storing actual file content. By combining these technologies, the system demonstrates how multiple cloud services can be integrated into a single working application.

2. Overall Approach

My application follows a client–server architecture. The frontend is responsible for user interaction, while the backend processes requests and communicates with external services. The system has clearly defined responsibilities:

- Authentication is handled by Firebase
- File storage is managed using Azure Blob Storage
- Application data (folders and metadata) is stored in MongoDB
- FastAPI acts as the backend controller that connects all components

These responsibilities improve the scalability, maintainability, and flexibility of the system.

The development process was completed in stages. Firstly, I implemented the directory structure and folder navigation. Next, I developed file upload, download, and deletion functionality. After that, I added duplicate file detection. Finally, I implemented advanced features such as search and sorting to improve usability.

3. System Architecture

The system consists of four main components:

3.1 Frontend

I developed the frontend using HTML and JavaScript. It provides the user interface that allows users to interact with the system. this includes forms for user authentication, folder creation, file upload, and search functionality.
The frontend also integrates Firebase authentication logic, which controls user login and generates an authentication token that is sent to the backend for verification.

3.2 Backend (FastAPI)

The backend is responsible for:

- Handling HTTP requests
- Managing user data
- Processing file operations
- Communicating with MongoDB and Azure

Every feature is implemented as a route (endpoint). These routes handle specific tasks such as uploading files or retrieving directory contents.

3.3 MongoDB

MongoDB stores user-related data, including:

- Email
- Folder structure
- File metadata

Each user has a root directory, which contains nested folders and files. this structure allows the application to simulate a file system.

3.4 Azure Blob Storage

Azure Blob Storage is used to store the actual files. Instead of storing files in the database, only metadata is stored in MongoDB, while file content is uploaded to Azure.

Each file is saved using a path that includes The user's email, the folder path, the file name

4. Authentication

Authentication is implemented using Firebase Authentication. When a user logs in, the following process occurs:

1. Firebase generates an authentication token
2. The token is stored in the user's browser as a cookie
3. The backend verifies the token for every incoming request

If the token is invalid or expired, the user is redirected with an error message prompting them to log in again. This approach ensures that:

- Only authenticated users can access the system
- User identity is securely verified

5. Key Functionalities

5.1 Folder Management

Users can create folders and navigate through them. the folder structure is stored as a nested object in MongoDB. the system allows

- Creating folders
- Navigating into folders
- Returning to the parent directory

The helper function for path handling ensures that the correct folder is accessed based on the current path.

5.2 File Upload

When a file is uploaded

1. The file is read by the backend
2. A hash value is generated for duplicate detection
3. The file is uploaded to Azure Blob Storage
4. Metadata is stored in MongoDB

The metadata includes file name, file size, file type, upload timestamp, hash value

5.3 Overwrite Protection

If a user uploads a file with the same name as an existing file:

- The system does not overwrite automatically
- Instead, the user is asked to confirm

Only when the user confirms does the file get replaced. This prevents accidental data loss and follows safe design practices.

5.4 File Download

Users can download files from Azure Blob Storage. The backend retrieves the file and sends it as a streaming response, allowing the browser to download it.

5.5 File Deletion

When a file is deleted It is removed from Azure storage and It is removed from MongoDB. this ensures consistency between stored data and metadata.

5.6 Duplicate Detection

Duplicate files are detected using hashing. Each file is assigned an MD5 hash when uploaded.

If two files have the same hash:

- They are considered duplicates
- The system highlights them in the user interface

This helps users identify redundant files easily.

5.7 Metadata Display

System displays additional file information such as file size, file type, upload date. this improves usability and gives users better control over their files.

5.8 Search and Sorting

Search and sorting features it allows users to search for files by name, sort files by name, size, or date. this functionality is handled in the backend and reflected in the UI.

6. How Components Work Together

The system operates as a processing workflow

1. The user interacts with the frontend
2. The request is sent to the FastAPI backend
3. The backend verifies the user's authentication
4. The backend retrieves or updates data in MongoDB
5. If file operations are required, Azure Blob Storage is used
6. A response is sent back to the frontend

This workflow ensures that all components interact efficiently while maintaining security and data consistency.

7. Challenges Faced

7.1 Token Expiration

One issue encountered was Firebase tokens expiring during use. this caused authentication failures. i solved this by catching token verification errors, displaying a clear message, redirecting users to log in again

7.2 Azure Storage Setup

Initially, there were issues connecting to Azure (Azurite). This was due to incorrect connection strings and container setup. I solved this problem by

- Correcting the connection string
- Ensuring the container is created at startup

7.3 Duplicate Detection

Detecting duplicate files required a reliable method. I handled this using MD5 hashing, which allows comparison of file content rather than just file names.

7.4 Overwrite Handling

At first, files were overwritten automatically. this was not good enough so I solved this by adding a confirmation steps in the UI, and adding backend validation

8. References

- MongoDB Inc. (2024). *MongoDB Documentation*. Available at: https://www.mongodb.com/docs/
- Microsoft Azure. (2024). *Azure Blob Storage Documentation*. Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/
- Google. (2024). *Firebase Authentication Documentation*. Available at: https://firebase.google.com/docs/auth
- FastAPI. (2024). *FastAPI Documentation*. Available at: https://fastapi.tiangolo.com/
- Python Software Foundation. (2024). *hashlib — Secure hashes and message digests*. Available at: https://docs.python.org/3/library/hashlib.html
- *Platform as a Service by Example* (Course Material), Griffith College

9. Conclusion

My project successfully demonstrates a cloud-based file management system that integrates multiple services. It provides essential features such as authentication, file storage, directory navigation, and duplicate detection. the system is modular, scalable, and user-friendly. It highlights how modern cloud technologies can be combined to build a fully functional application.

10. GitHub Repository

My GitHub repository link is:
https://github.com/Toluwani9-ai/cloud-file-manager-assignment